



국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

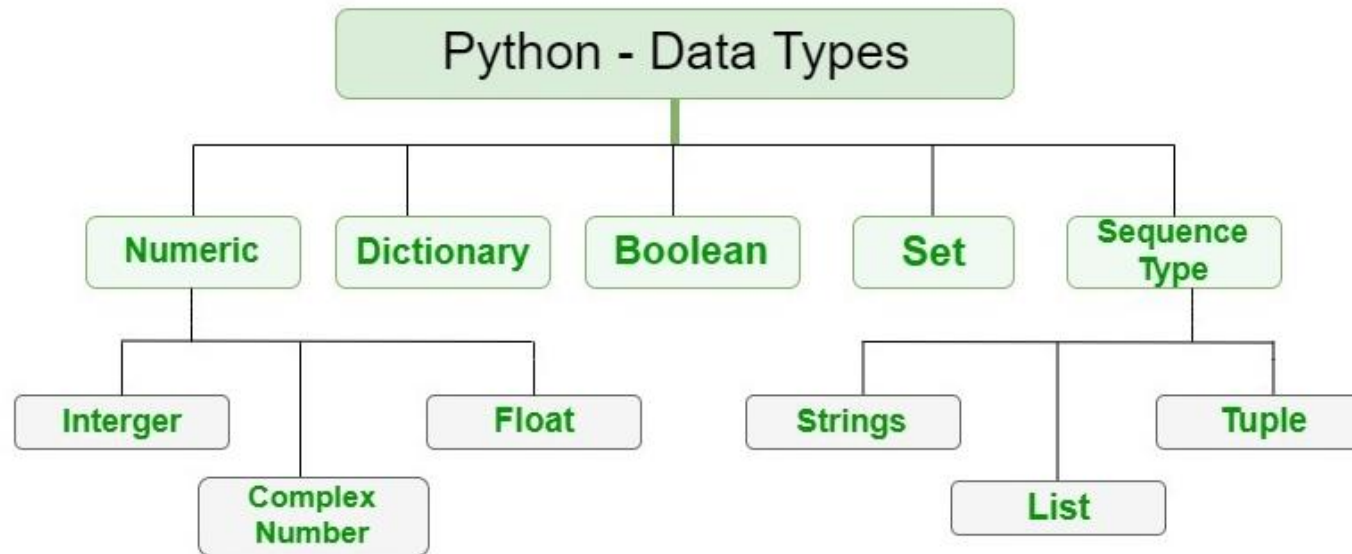
친환경스마트조선기자재학과

데이터처리특론



자료형

- 컴퓨터 과학과 프로그래밍 언어에서 실수, 정수, 불린 자료형 따위의 여러 종류의 데이터를 식별하는 분류
- 더 나아가 해당 자료형에 대한 가능한 값, 해당 자료형에서 수행을 마칠 수 있는 명령들, 데이터의 의미, 해당 자료형의 값을 저장하는 방식을 결정한다.



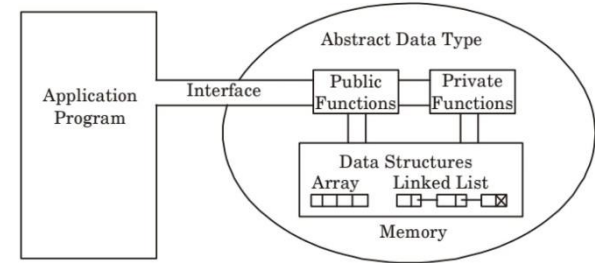
Python Data Structures

ADT(Abstract Data Type)



추상 자료형(abstract data type, ADT)

- 자료들과 그 자료들에 대한 연산들을 명기한 것
- 추상 자료형은 구현 방법을 명시하고 있지 않다는 점에서 자료 구조와 다르다.
- 인터페이스와 구현을 분리하여 추상화 계층을 둔 것이다.
- 예를 들어 전기 밥솥을 추상 자료형에 비유한다면 그 속에 들어가는 밥은 자료가 되고, 밥솥에 있는 취사, 예약취사 버튼들과 남은 시간을 표시하는 디스플레이에 어떤 내용들이 표시되어야 하는지를 명기한 것이다.
- 추상 자료형에서는 이것들이 어떻게 구성되는지 관심이 없고, 몇 와트의 전기를 소모하는 지에 대해서도 관심이 없다.
- 자료에 대한 연산은 자료에 대하여 질의를 던지는 것과 자료를 변경하기 위한 연산으로 나뉜다.



List ADT(Abstract Data Type)

List ADT

- 리스트는 순서가 있는 연속적인 데이터 집합이다.
- 리스트에 저장된 각각의 데이터들을 요소(element)라고 부른다.
- 요소가 없는 빈 리스트도 가능하다.
- 리스트에 저장된 요소의 개수를 리스트의 길이라고 하는데 size 변수에 저장한다.
- 리스트 뒤에 요소를 추가하는 것을 append라고 한다.
- 리스트의 요소를 찾아 지우는 것을 remove라고 한다.
- 리스트 정체를 비우는 것을 clear라고 한다.
- 등등등

i번 자리에 원소 x를 삽입한다
i번 원소를 삭제한다
원소 x를 삭제한다
i번 원소를 알려준다
원소 x가 몇 번 원소인지 알려준다
리스트의 사이즈(원소의 총 수)를 알려준다

List

- empty list has size 0
- insert
- remove
- Count
- Read/modify element at a position
- Specify data-type

리스트(list) 또는 선형 리스트(linear list)

- '줄 세워져 있는 데이터' 또는 '죽어선 데이터'
- 자료들이 차례대로 나열된 선형 자료구조
- 각 자료는 순서 또는 위치(position)를 가짐
- 순서가 있는 수정 가능한 객체의 집합입니다.
- 수정, 삭제, 추가가 가능합니다.
- list 는 [] 대괄호로 작성되어지며, 내부 원소는 ,로 구분됩니다.



리스트(배열 리스트) 객체 구조

insert()	
append()	
pop()	
remove()	
index()	
clear()	
count()	
extend()	
copy()	
reverse()	
sort()	

insert(i, x)	◀ x를 리스트의 i번 원소로 삽입한다.(맨 앞자리는 0번)
append(x)	◀ 원소 x를 리스트의 맨 뒤에 추가한다.
pop(i)	◀ 리스트의 i번 원소를 삭제하면서 알려준다.
remove(x)	◀ 리스트에서 (처음으로 나타나는) x를 삭제한다.
index(x)	◀ 원소 x가 리스트의 몇 번 원소인지 알려준다.
clear()	◀ 리스트를 깨끗이 청소한다.
count(x)	◀ 리스트에서 원소 x가 몇 번 나타나는지 알려준다.
extend(a)	◀ 리스트에 나열할 수 있는 객체(예 리스트) a를 풀어서 추가한다.
copy()	◀ 리스트를 복사한다.
reverse()	◀ 리스트의 순서를 역으로 뒤집는다.
sort()	◀ 리스트의 원소들을 정렬한다.

삽입

10	35	40	17	95	50	48	33	9			
----	----	----	----	----	----	----	----	---	--	--	--

↑
3번 원소

↑
8번 원소



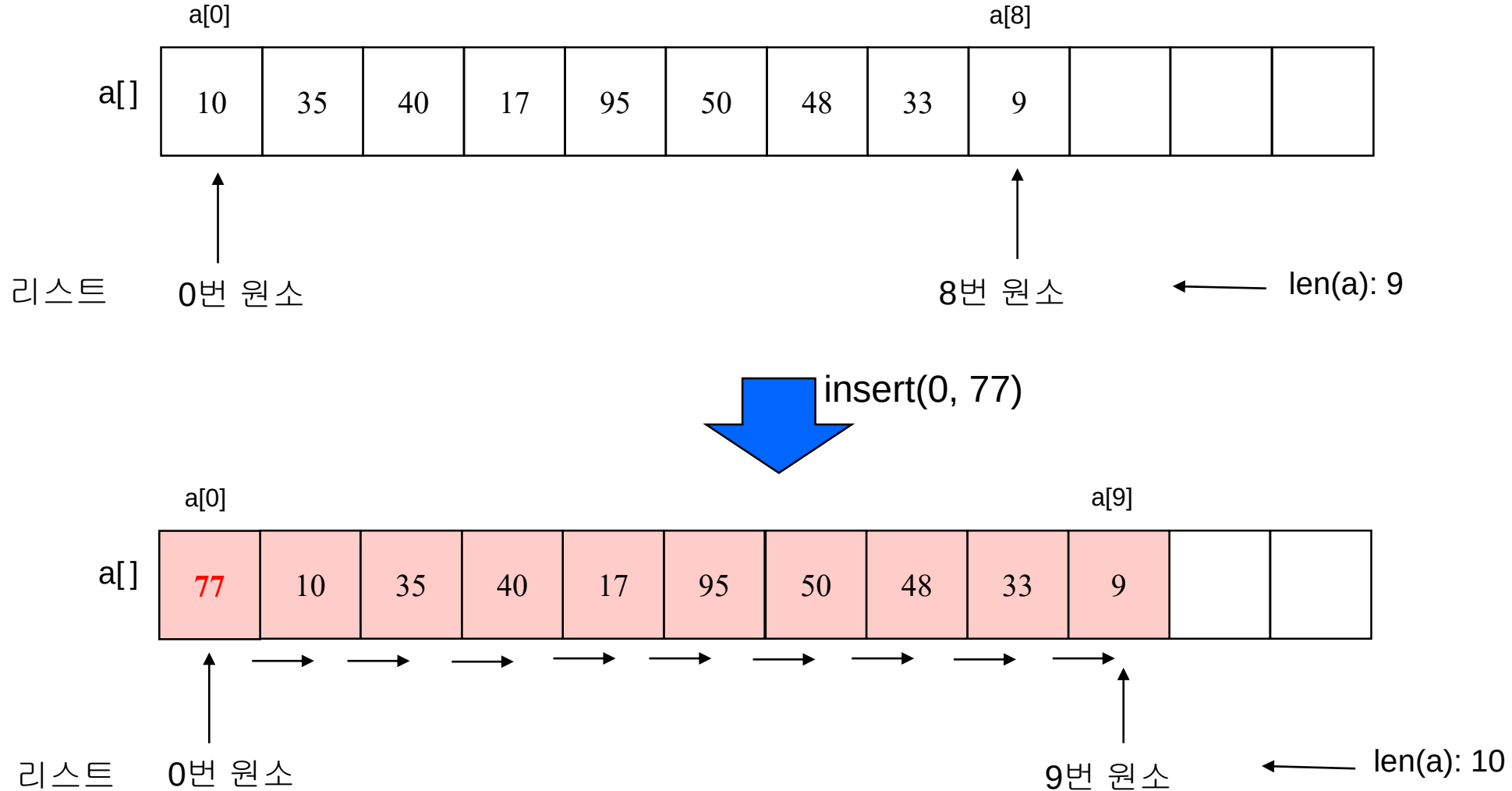
add(3, 77)

10	35	40	77	17	95	50	48	33	9		
----	----	----	----	----	----	----	----	----	---	--	--

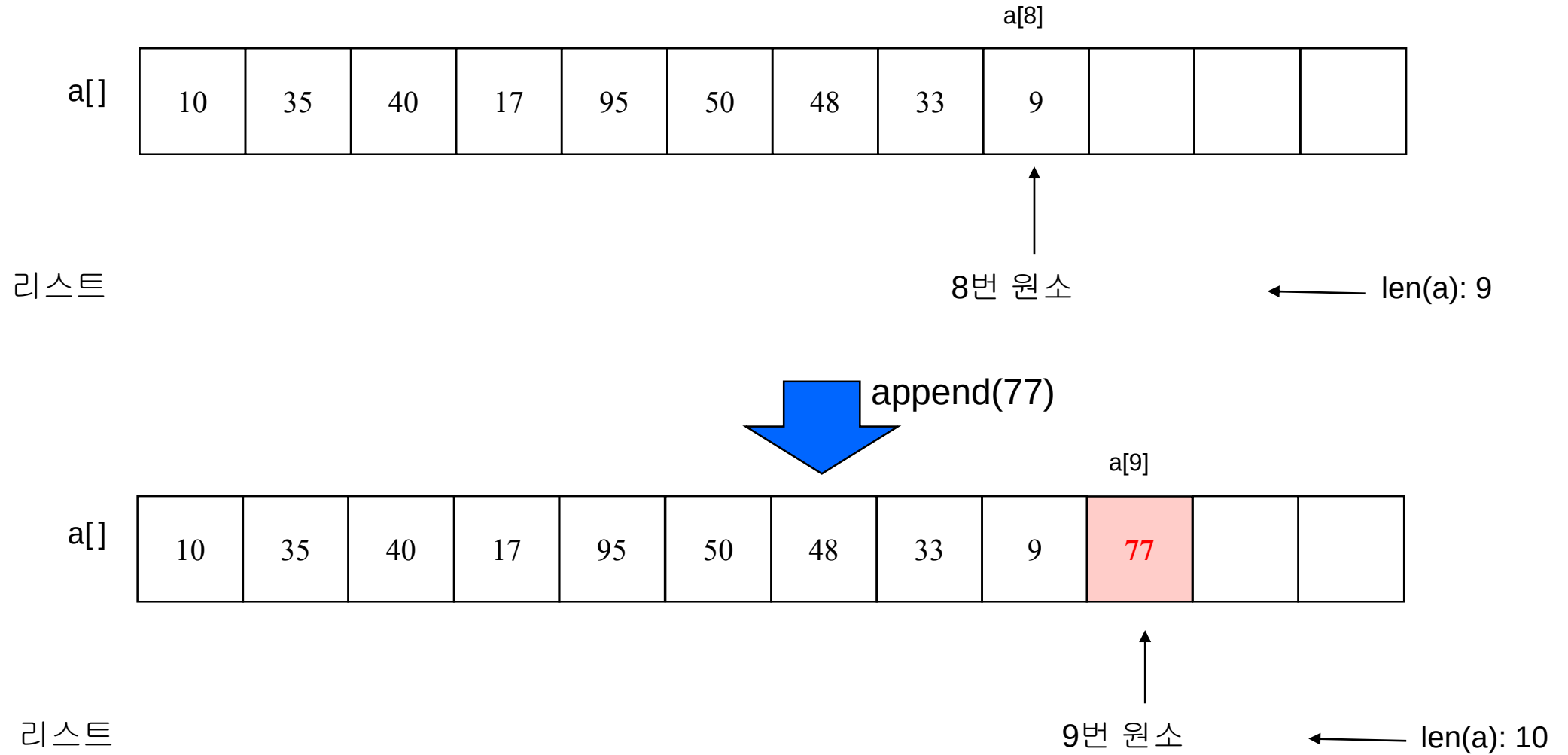
↑ → → → → →
3번 원소

↑
9번 원소

삽입 효율이 가장 나쁜 예



삽입 효율이 가장 좋은 예



삭제

10	35	40	17	95	50	48	33	9			
----	----	----	----	----	----	----	----	---	--	--	--

↑
3번 원소

↑
8번 원소



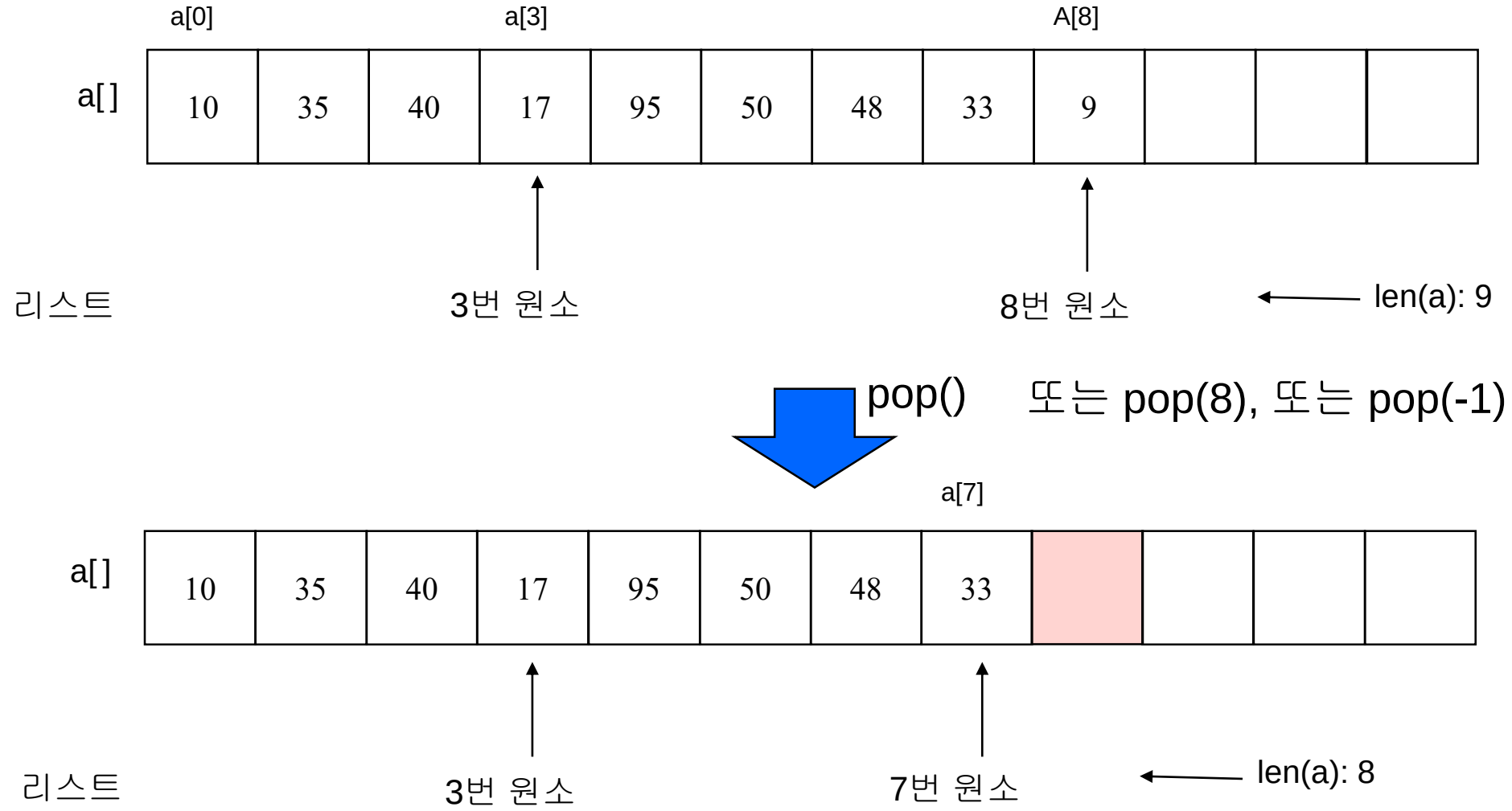
pop(3)

10	35	40	95	50	48	33	9				
----	----	----	----	----	----	----	---	--	--	--	--

↑ ← ← ←
3번 원소

↑ ← ←
7번 원소

맨 끝 원소 삭제 방법은 3가지

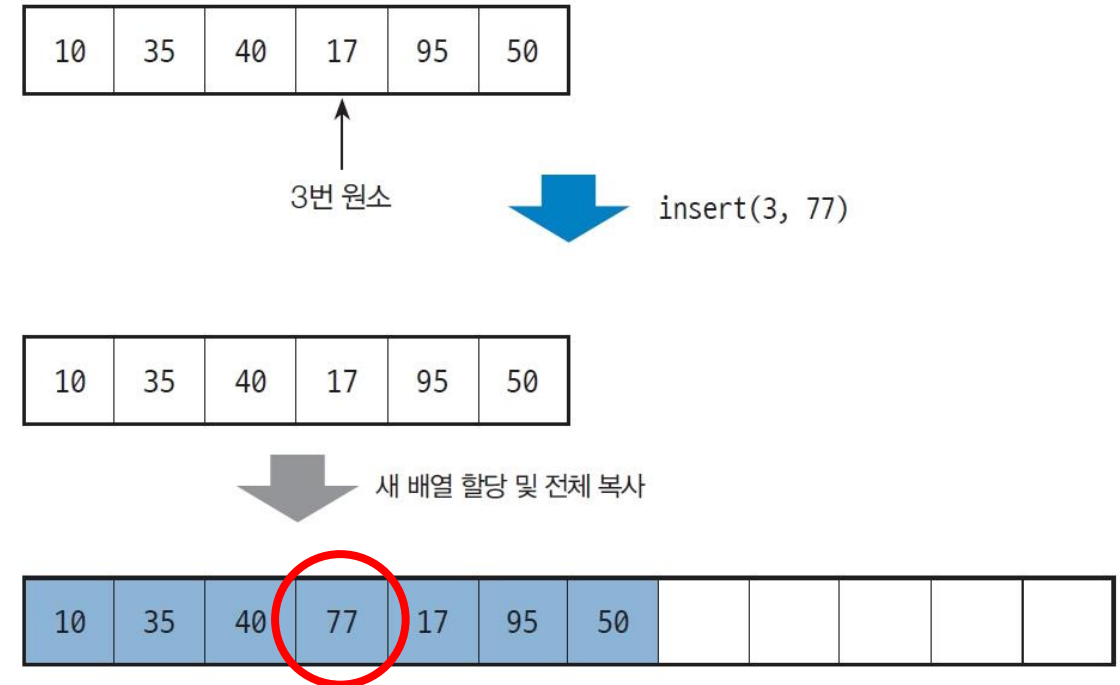
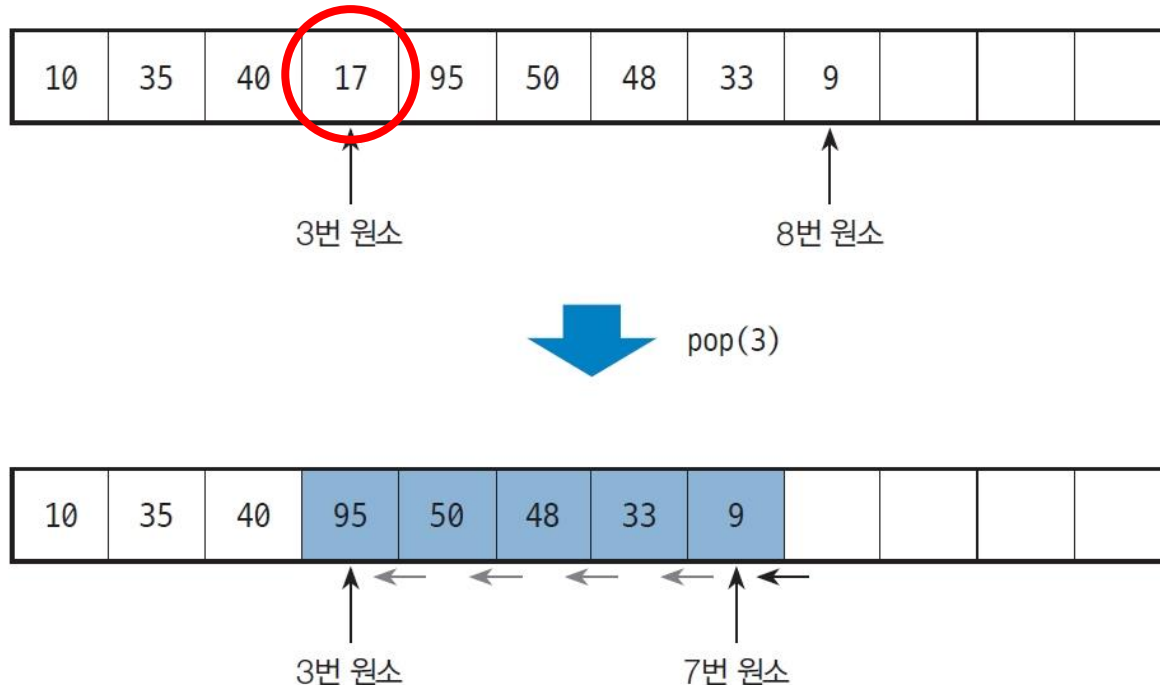


파이썬 내장 리스트의 한계

배열의 단점

- 연속된 공간에 저장되므로 삽입이나 삭제 시 시프트 작업이 필요하다
- 미리 크기를 정해야 하므로 오버플로우 시 배열을 새로 배정받아 내용을 복사해야 한다

But, 파이썬에서 자동으로 관리해주어 사용자가 이를 신경 쓸 필요는 없다



메서드 사용 방법

일반 list

```
>>> [1, 1, 2, 3, 5, 8]
[1, 1, 2, 3, 5, 8]
>>> a = [1, 1, 2, 3, 5, 8]
>>> type(a)
<class 'list'>
```

index 범위를 벗어나면 에러가 발생

```
>>> a[6]
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    a[6]
IndexError: list index out of range
```

list는 0부터 시작하는 인덱스로 접근가능

```
>>> a[0]
1
>>> a[5]
8
```

여러가지 타입이 리스트에 포함

```
>>> seq = ['a', 456, True, [1, 2, 3]]
>>> seq
['a', 456, True, [1, 2, 3]]
```

메서드 사용 방법

빈 리스트 만들기

```
>>> b=list()
>>> c=[]
>>> b
[]
>>> c
[]
```

리스트에 원소 추가

```
>>> b.append(5)
>>> b.append(1.4)
>>> b
[5, 1.4]
```

리스트 합치기

```
>>> [1, 3, 5, 7] + [2, 4, 6]
[1, 3, 5, 7, 2, 4, 6]
```

문자열을 리스트로 변형

```
>>> list("가나다라")
['가', '나', '라', '다', '라']
>>> strSeq = list("가나다라")
>>> strSeq
['가', '나', '다', '라']
```

마지막 원소뒤에 콤마를 남겨도
에러가 나지 않음

```
>>> ['가',
'나',
'다',
'라',
]
['가', '나', '다', '라']
```

list indexing(리스트 인덱싱)

- 파이썬에서 리스트 인덱싱은 -(음수 인덱싱) 값도 허용한다.
- - 값은 역순으로도 인덱싱된다.
- 다른 언어에서는 마지막 열을 찾기 위해 리스트의 길이를 구해서 찾아야 하지만, 파이썬은 -1만으로도 손쉽게 찾을 수 있다.

```
>>> s = 'show how to index into sequences'.split()
>>> s
['show', 'how', 'to', 'index', 'into', 'sequences']
>>> s[0]
'show'
>>> s[5]
'sequences'
>>> s[-1]
'sequences'
>>> s[-2]
'into'
```

list slicing(리스트 슬라이싱)

- 사용법 : 리스트변수[시작인덱스:종료인덱스:step]
- 종료인덱스의 원소는 포함되지 않고 바로 앞 원소까지만 포함
- step은 생략가능

<pre>s='show how to index into sequences'.split() s s[1:4] s[1:-1] s[3:] s[:3] s[:3] + s[3:] == s full_slice = s[:] full_slice full_slice == s full_slice is s u = s.copy() v = list(s) s[::2] s[::-1]</pre>	<pre>#문자열을 리스트로 #s 리스트 출력 #시작 인덱스와 종료 인덱스 #음수 인덱싱도 슬라이싱에 사용 #시작인덱스부터 끝까지 포함 #처음부터 특정인덱스까지 가져오기 #동일한 리스트 #모든 값을 복사하여 새로운 리스트 만들기 #값은 같지만 같은 변수는 아님 #리스트 복사 #리스트 복사 #step 사용 #리스트 reverse</pre>
--	---

mutable과 immutable 객체

class	설명	구분
list	mutable 한 순서가 있는 객체 집합	mutable
set	mutable 한 순서가 없는 고유한 객체 집합	mutable
dict	key와 value가 맵핑된 객체, 순서 없음	mutable
bool	참,거짓	immutable
int	정수	immutable
float	실수	immutable
tuple	immutable 한 순서가 있는 객체 집합	immutable
str	문자열	immutable
frozenset	immutable한 set	immutable

Immutable과 Mutable

shallow copy deep copy

```
>>> a == b
True
>>> a is b
False
>>> b[0] = 5
>>> a
[1, 2, 3]
>>> b
[5, 2, 3]
```

```
>>> import copy
>>> a = [[1,2],[3,4]]
>>> b = copy.deepcopy(a)
>>> a[1].append(5)
>>> a
[[1, 2], [3, 4, 5]]
>>> b
[[1, 2], [3, 4]]
```

```
>>> import copy
>>> a = [[1,2],[3,4]]
>>> b = copy.copy(a)
>>> a[1].append(5)
>>> a
[[1, 2], [3, 4, 5]]
>>> b
[[1, 2], [3, 4, 5]]
```

shallow copy deep copy

Assignment	Shallow Copy	Deep Copy
<code>B = A</code>	<code>C = copy(A)</code>	<code>D = deepcopy(A)</code>
<code>id(A) == id(B)</code>	<code>id(A) != id(C)</code>	<code>id(A) != id(D)</code>
<code>id(A[i]) == id(B[i])</code>	<code>id(A[i]) == id(C[i])</code>	<code>id(A[i]) != id(D[i])</code> (exceptions apply)

The Python Memory Model

대표적인 list 관련 메소드

Python List methods

Method	Description
<u>append</u> (.).	Adds an element at the end of the list
<u>clear</u> (.).	Removes all the elements from the list
<u>copy</u> (.).	Returns a copy of the list
<u>count</u> (.).	Returns the number of elements with the specified value
<u>extend</u> (.).	Add the elements of a list (or any iterable), to the end of the current list
<u>index</u> (.).	Returns the index of the first element with the specified value
<u>insert</u> (.).	Adds an element at the specified position
<u>pop</u> (.).	Removes the element at the specified position
<u>remove</u> (.).	Removes the first item with the specified value
<u>reverse</u> (.).	Reverses the order of the list
<u>sort</u> (.).	Sorts the list

클래스

- 클래스는 객체의 구조와 행동을 정의
- 객체의 클래스는 초기화를 통해 제어
- 클래스는 복잡한 문제를 다루기 쉽도록 만듦
- class 키워드 사용하여 새로운 클래스를 작성
- 클래스의 네이밍 컨벤션은 CamelCase 를 사용

```
class CustomClass:  
    def __init__(self, param):  
        .....
```

Camel case, Kebab case, Snake case 그리고 Pascal case

Class Example

```
# OOP (Class)
class Car:
    def __init__(self, speed, color, model):
        self.speed = speed
        self.color = color
        self.model = model

    def drive(self):
        self.speed = 50

myCar = Car(0, "green", "testCar")
print("-----Car Object Create Complete-----")
print("Car Speed: ", myCar.speed)
print("Car Color: ", myCar.color)
print("Car Model: ", myCar.model)
```

```
-----Car Object Create Complete-----
Car Speed: 0
Car Color: green
Car Model: testCar
'''

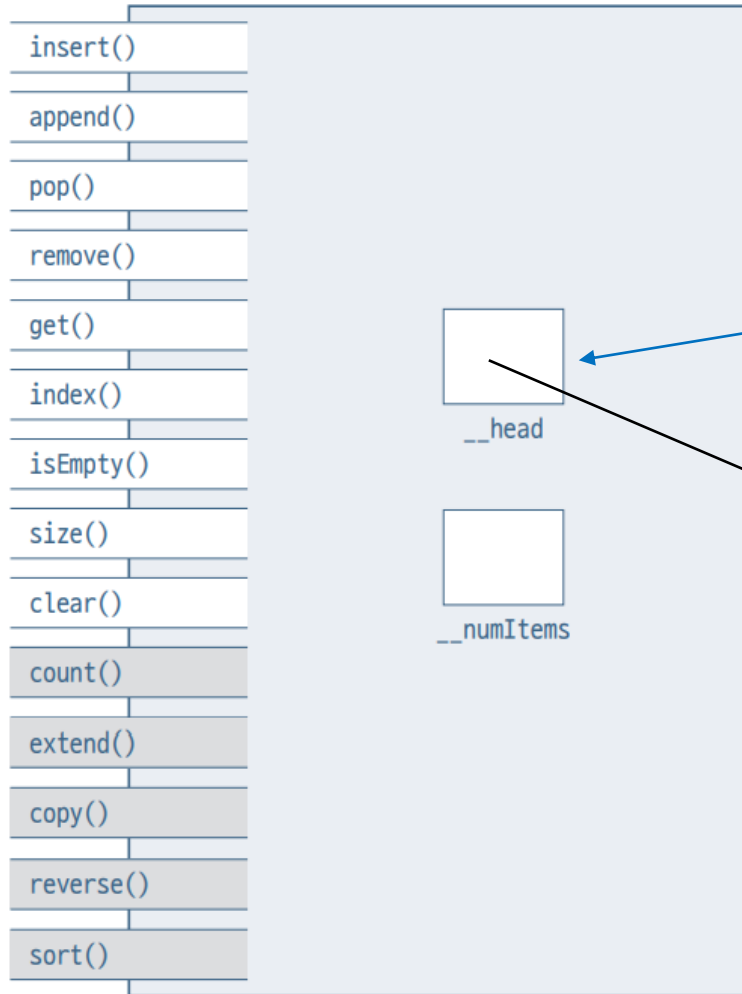
# 운전 중이 아니므로 speed는 0을 출력한다.
print("Car Speed by drive: ", myCar.speed)
'''

Car Speed by drive: 0
'''

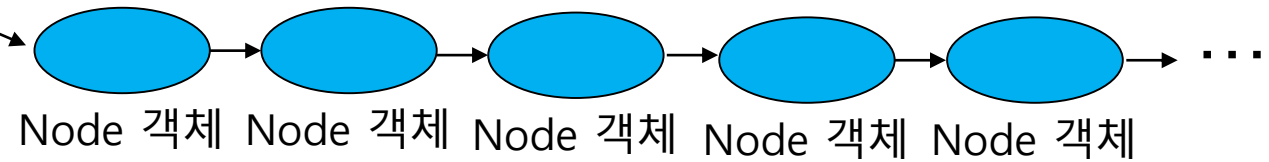
# Car object method Call (메서드 호출)
myCar.drive()
print("Car Speed by drive: ", myCar.speed)
'''

Car Speed by drive: 50
```

Linked List 객체 구조



Linked list의
시작 노드 레퍼런스만 있으면 된다



Linked List

Linked List 객체

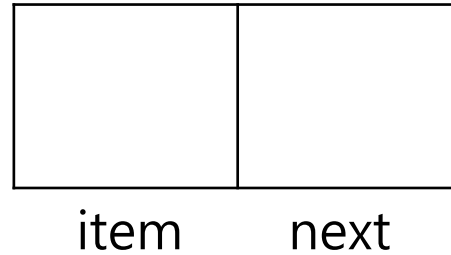
각 작업의 의미

<code>__head</code>	◀ 첫 번째 노드에 대한 레퍼런스
<code>__numItems</code>	◀ 연결 리스트에 들어 있는 원소의 총 수
<code>insert(i, x)</code>	◀ x를 연결 리스트의 i번 원소로 삽입한다. (맨 앞자리는 0번)
<code>append(x)</code>	◀ 연결 리스트의 맨 뒤에 원소 x를 추가한다.
<code>pop(i)</code>	◀ 연결 리스트의 i번 원소를 삭제하면서 알려준다.
<code>remove(x)</code>	◀ 연결 리스트에서 (처음으로 나타나는) x를 삭제한다.
<code>get(i)</code>	◀ 연결 리스트의 i번 원소를 알려준다.
<code>index(x)</code>	◀ 원소 x가 연결 리스트의 몇 번 원소인지 알려준다.
<code>isEmpty()</code>	◀ 연결 리스트가 빈 리스트인지 알려준다.
<code>size()</code>	◀ 연결 리스트의 총 원소 수를 알려준다
<code>clear()</code>	◀ 연결 리스트를 깨끗이 청소한다.
<code>count(x)</code>	◀ 연결 리스트에서 원소 x가 몇 번 나타나는지 알려준다.
<code>extend(a)</code>	◀ 연결 리스트에 나열할 수 있는 객체(예 리스트, 튜플 등) a를 풀어서 추가한다.
<code>copy()</code>	◀ 연결 리스트를 복사해서 새 연결 리스트를 리턴한다.
<code>reverse()</code>	◀ 연결 리스트의 순서를 역으로 뒤집는다.
<code>sort()</code>	◀ 연결 리스트를 정렬한다.

ListNode 객체 구조



보통은 이렇게 그린다

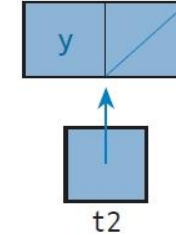


노드 클래스

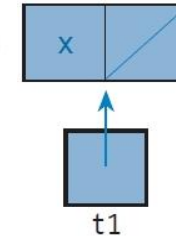
```

class ListNode:
    def __init__(self, newItem, nextNode:'ListNode'):
        self.item = newItem
        self.next = nextNode
    
```

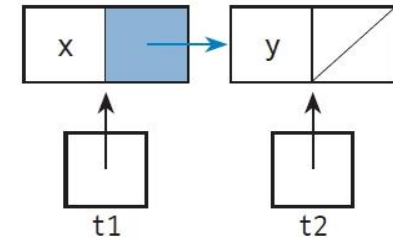
(a) `t2 = ListNode(y, None)`



(b) `t1 = ListNode(x, None)`



(c) `t1.next = t2`



(d) `t3 = ListNode(z, t1)`

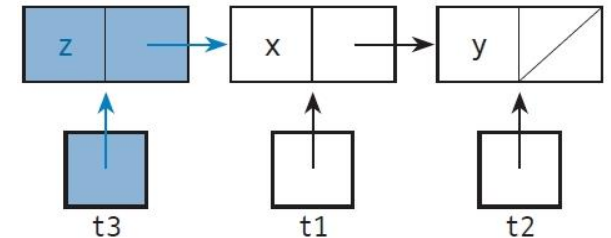
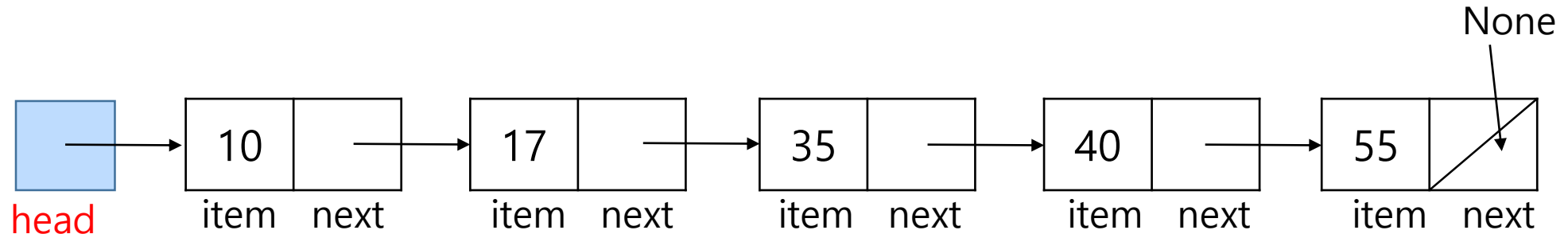


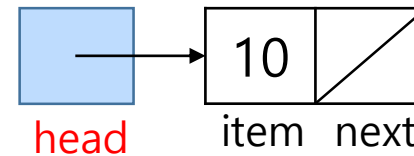
그림 5-19 클래스 ListNode의 사용 예

헤드 노드 Head Node

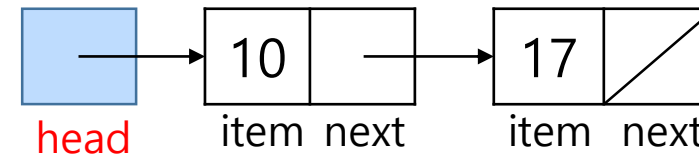
연결 리스트는 흔히 첫 노드에 대한 레퍼런스(여기서는 head)를 갖고 있다.



`head = ListNode(10, None)`



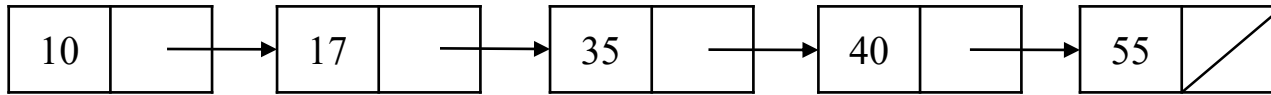
`head.next = ListNode(17, None)`



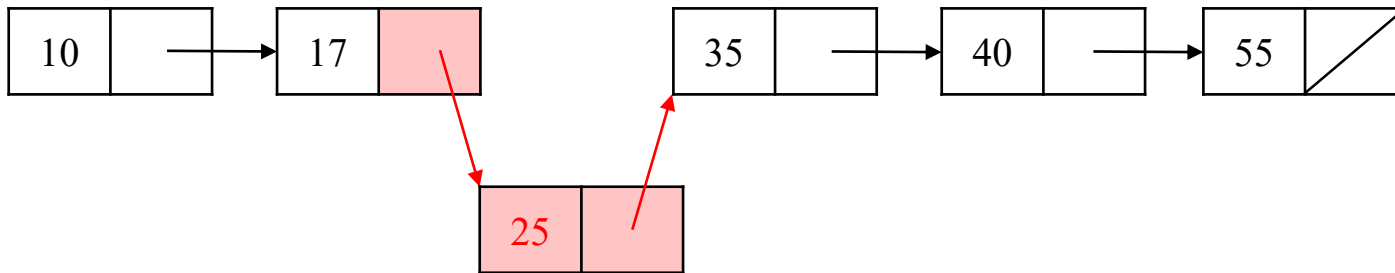
✓ 여기서, head는 단순한 레퍼런스 변수

핵심 작업

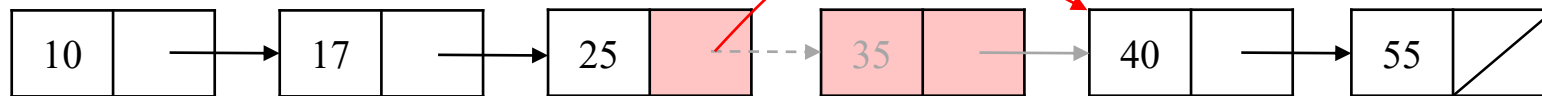
정수의 연결 리스트



삽입

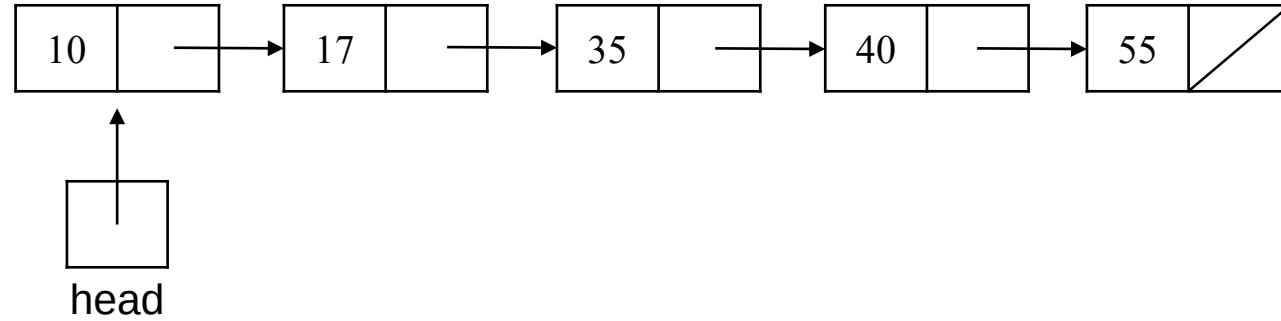


삭제

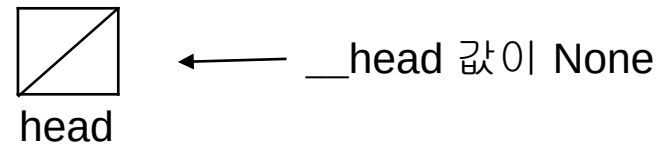


전형적 모양

대표적 상태

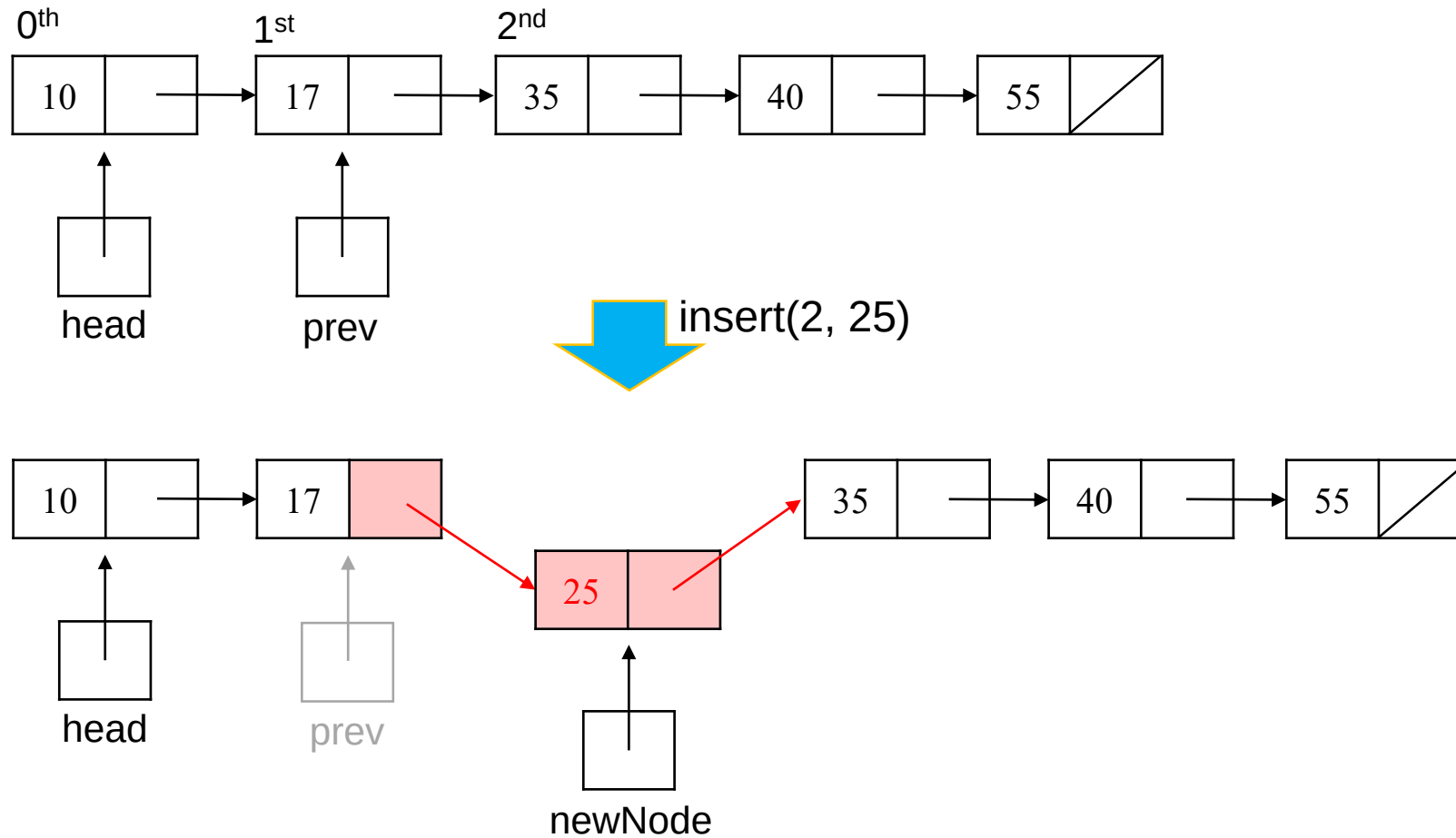


초기 상태



삽입Insertion

prev 다음에 새 노드를 삽입한다

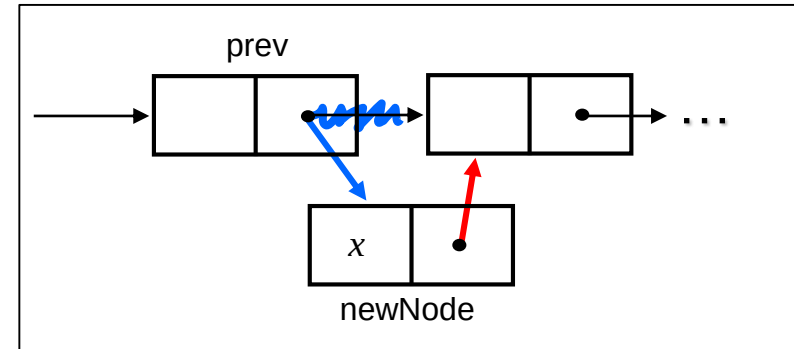


insert()

```

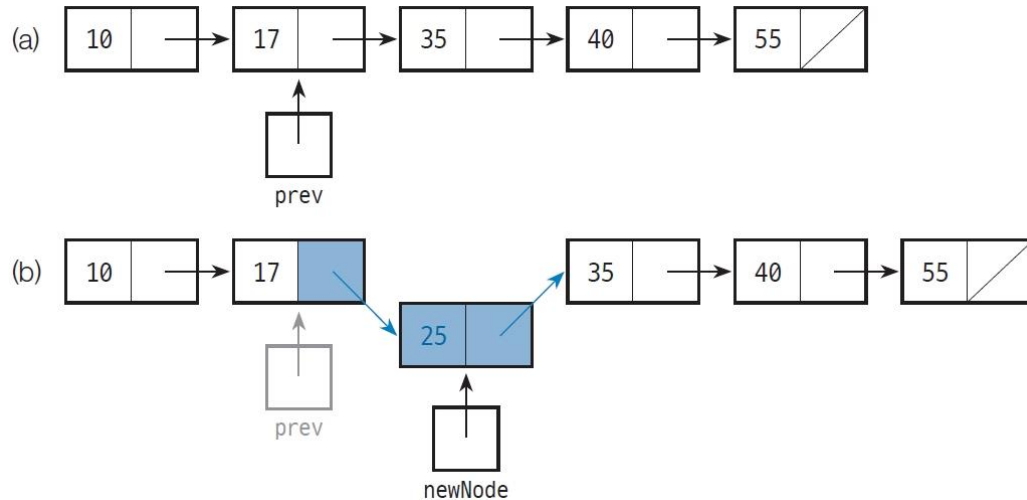
newNode.item ← x
newNode.next ← prev.next
prev.next ← newNode
__numItems += 1
    
```

중간 삽입과 맨 끝 삽입은 문제 없다.
 But, 맨 앞 삽입은 작동하지 않는다.

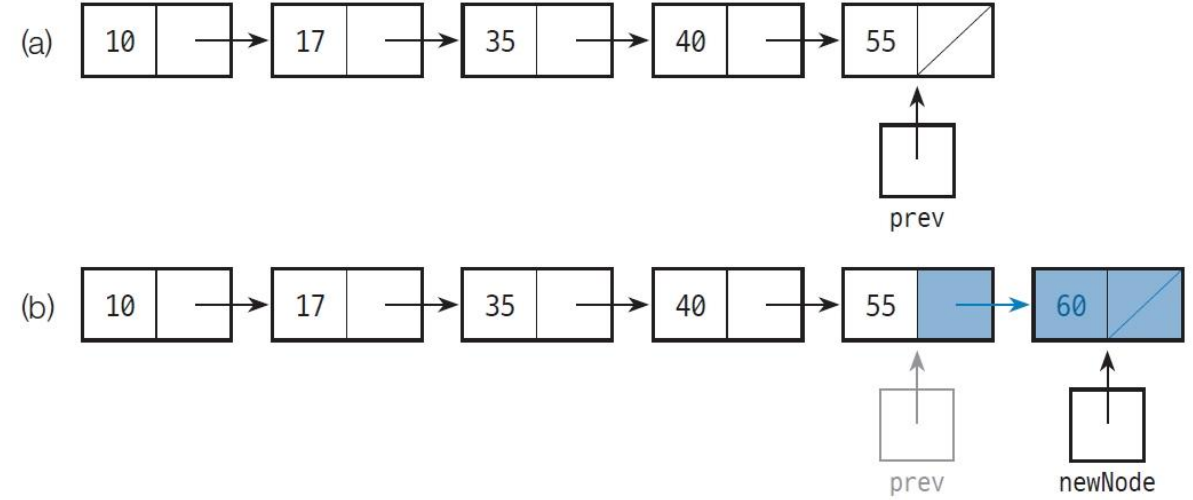


Animation

중간에 삽입: Okay



맨 끝에 삽입: Okay



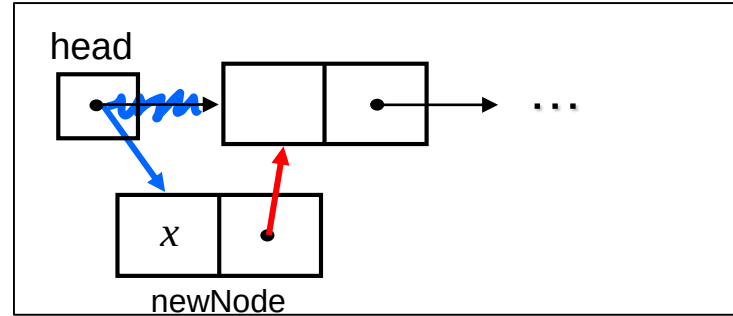
insert()

```

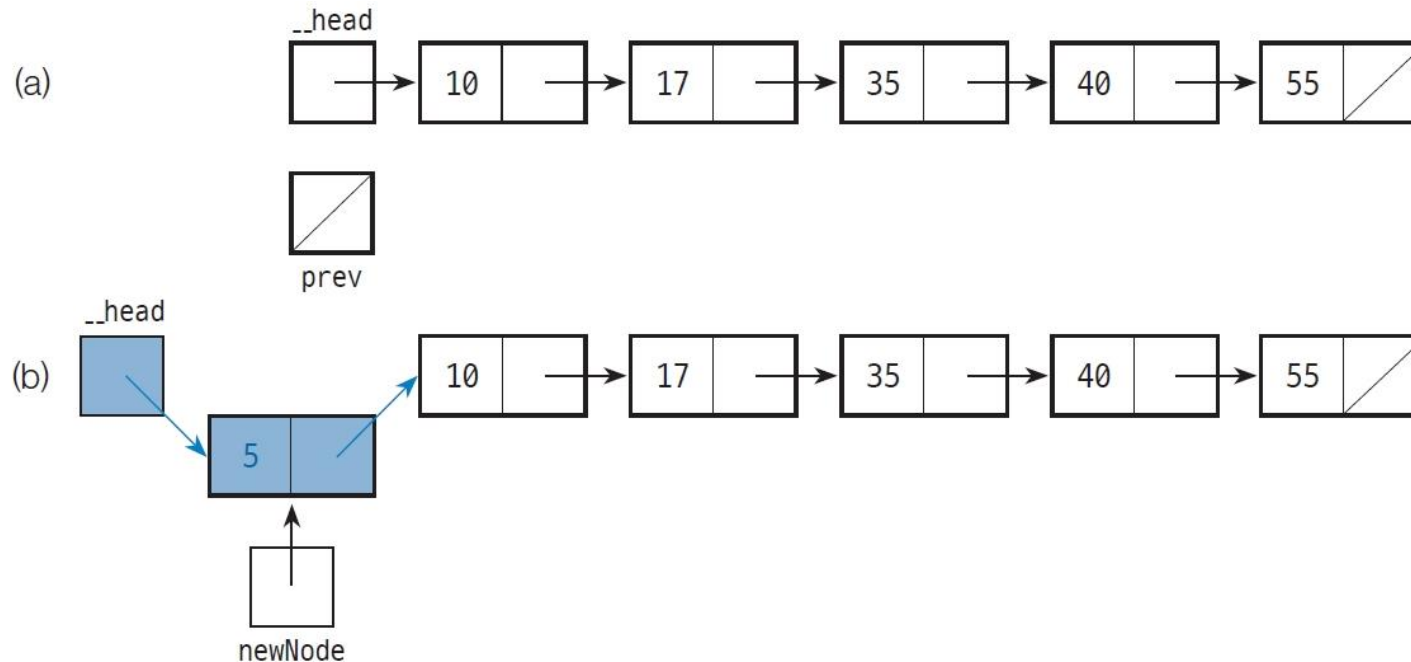
newNode.item ← x
newNode.next ← head
head ← newNode
numItems++
    
```

맨 앞에 삽입할 때

prev가 존재하지 않아 prev.next와 같은 표현 사용불가



Animation



insert()

알고리즘 5-1 연결 리스트에 원소 삽입하기

```
if i == 0:
    newNode.item = x
    newNode.next = __head
    __head = newNode
    __numItems += 1
else:
    newNode.item = x
    newNode.next = prev.next
    prev.next = newNode
    __numItems += 1
```

← 앞의 두 가지 경우를 다 고려한

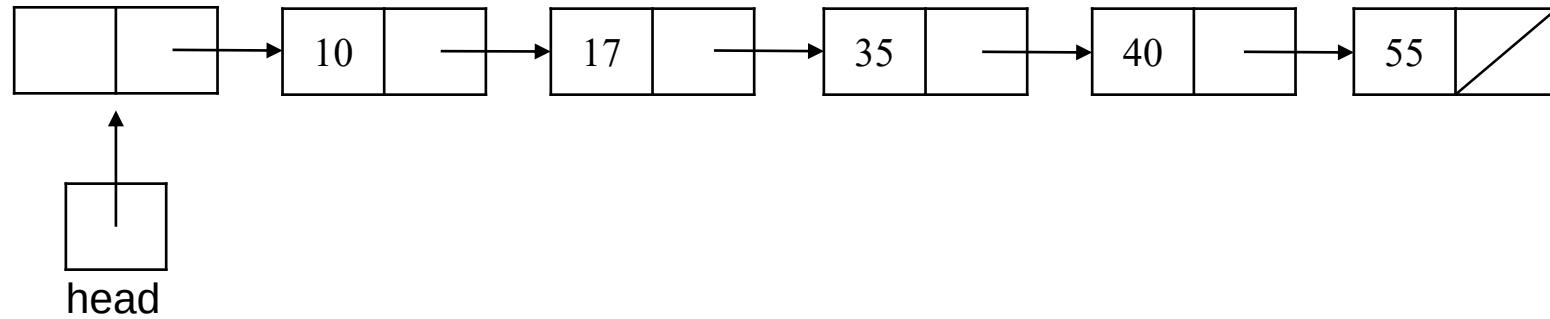
자주 나오지 않는 “맨 앞 삽입” 때문에
이렇게 항상 두 가지 경우로 나누어 처리해야 하는가?

하나로 처리할 수 있는 방법이 있다 → 더미 헤드 노드 dummy head node

더미 헤드를 둔 연결 리스트

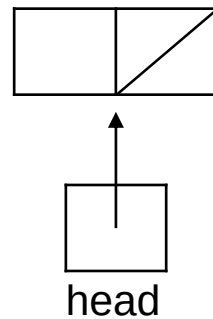
대표적 상태

dummy head

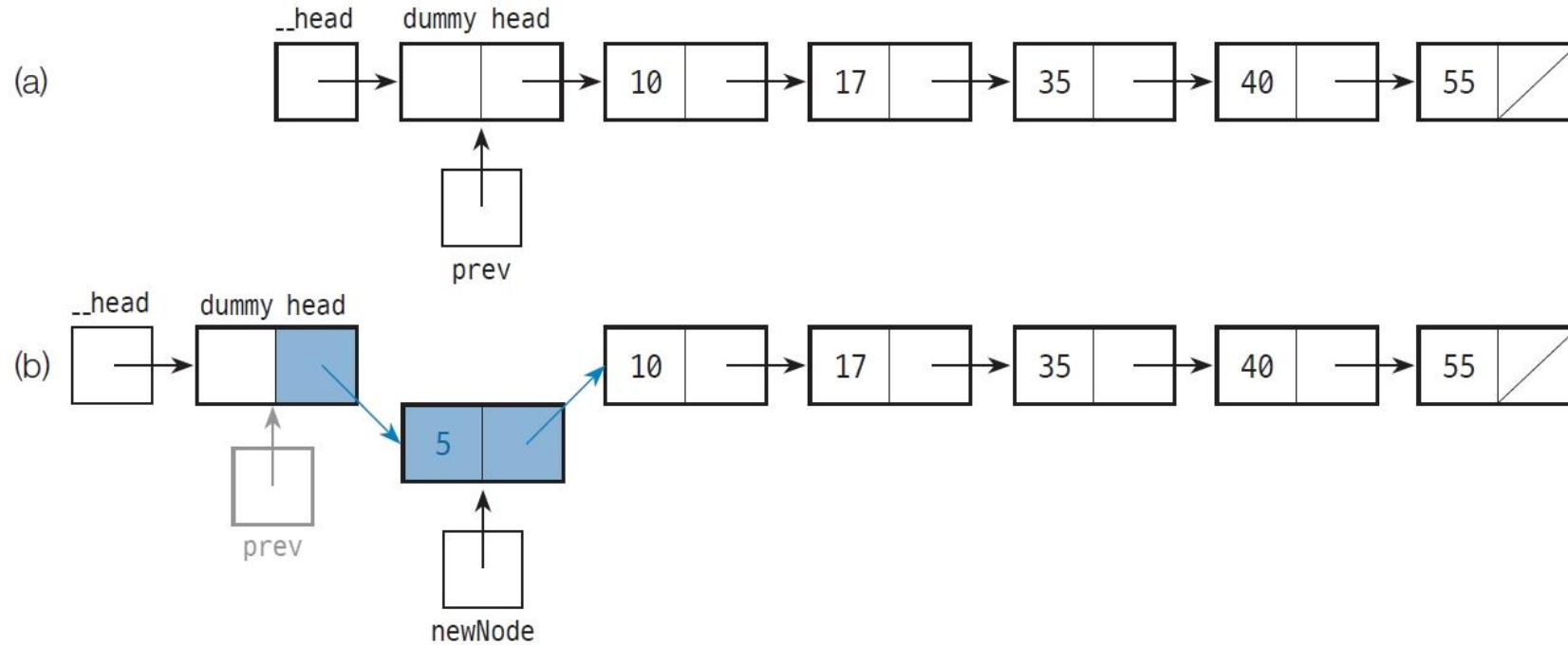


초기 상태

dummy head



insert(): 더미 헤드가 있는 버전



알고리즘 5-2 연결 리스트에 원소 삽입하기(더미 헤드를 두는 대표 버전)

```

newNode.item = x
newNode.next = prev.next
prev.next = newNode
__numItems += 1
    
```

← Dummy head를 두면 모든 삽입은 이것으로 충분
 항상 prev가 존재하기 때문

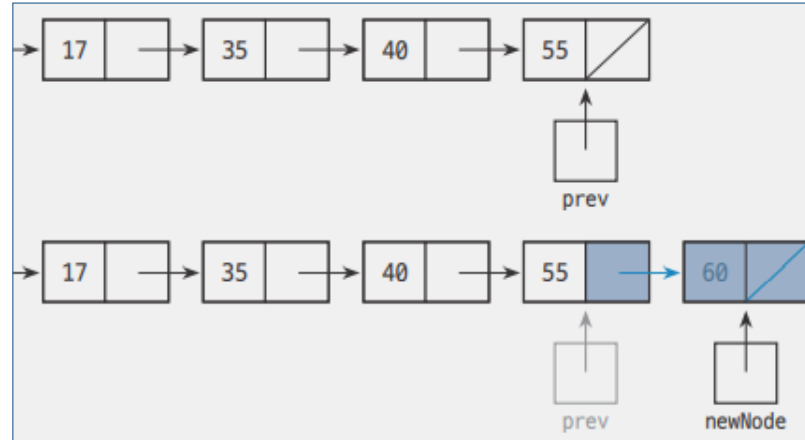
append()

맨 뒤에 원소 x 를 추가

```

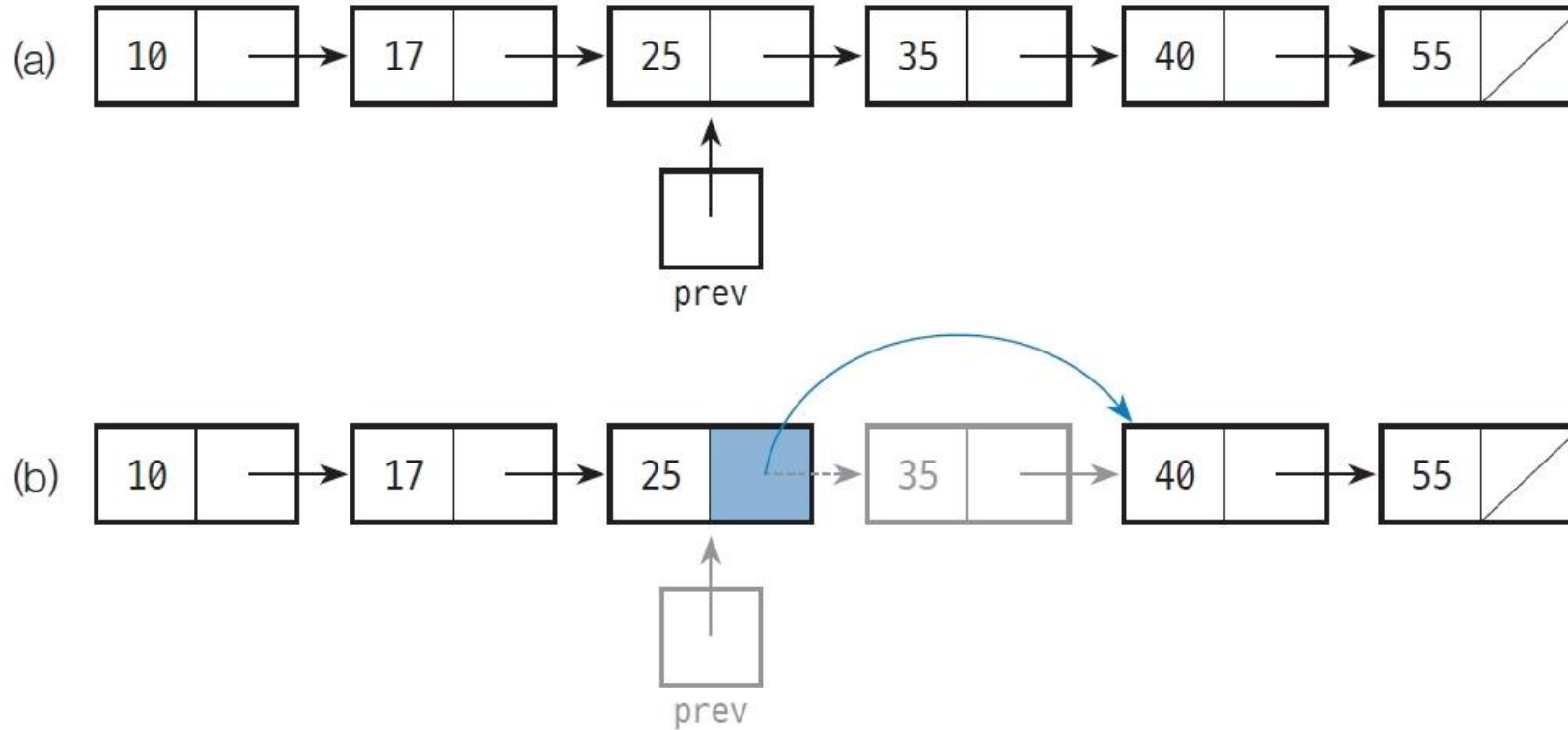
newNode.item ← x
newNode.next ← prev.next
prev.next ← newNode
__numItems += 1
    
```

← prev가 정해지면
insert()의 코드와 동일



삭제 Deletion

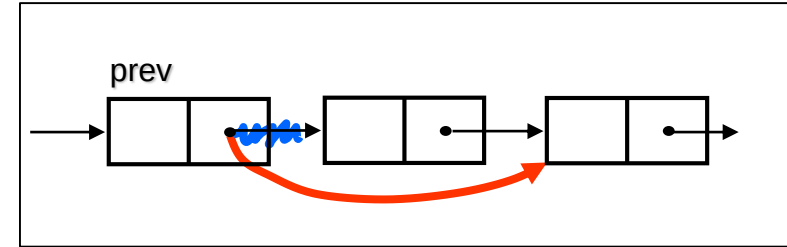
prev 다음 노드를 삭제한다



pop()

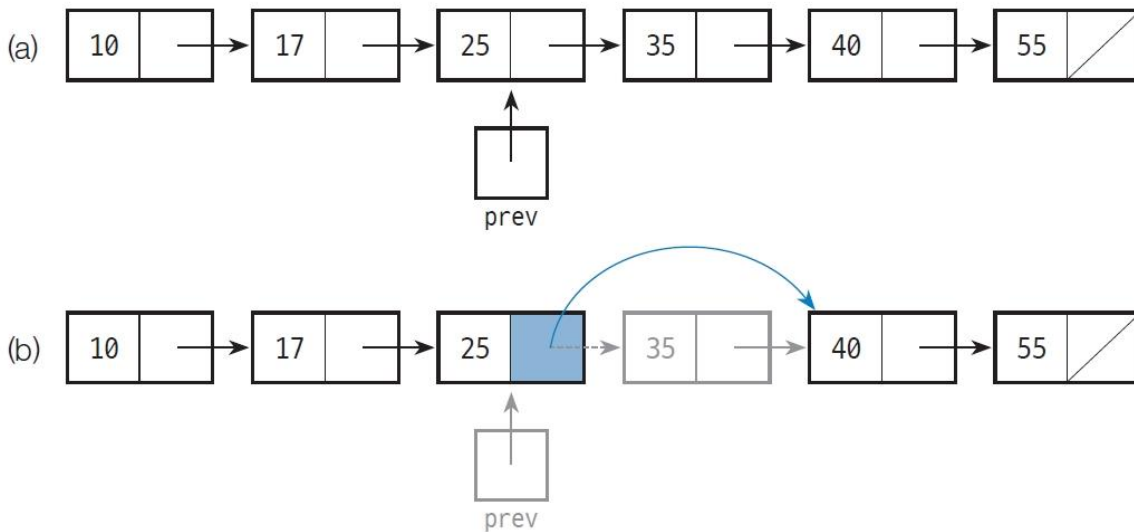
$\text{prev.next} \leftarrow \text{prev.next.next}$
 numItems--

중간 노드 삭제와 맨 끝 노드 삭제는 Okay
 But, 맨 앞 노드 삭제는 작동하지 않는다

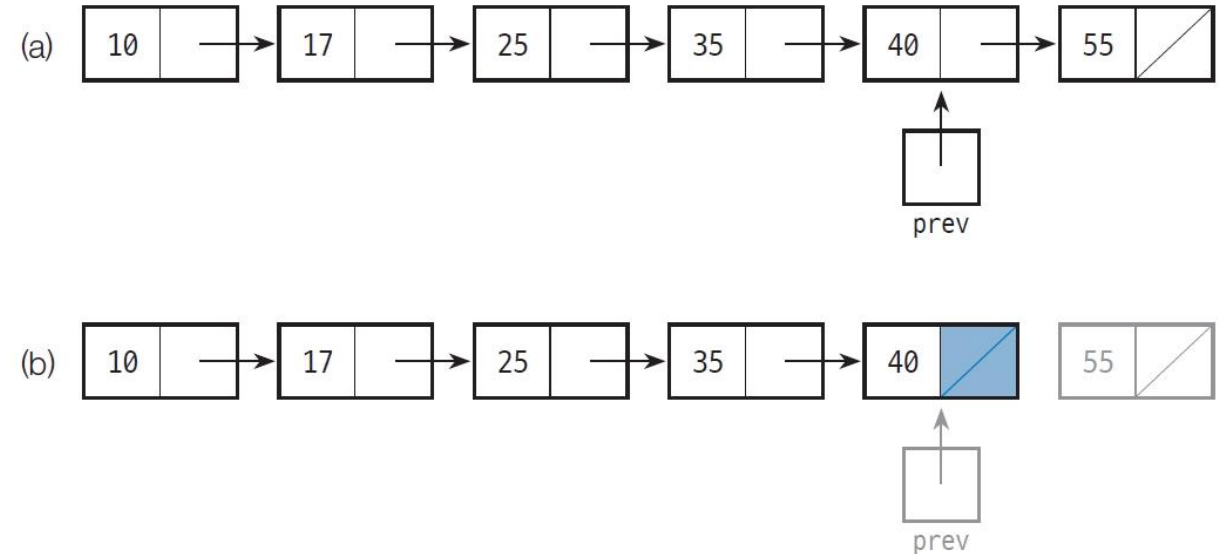


Animation

중간 노드 삭제: Okay

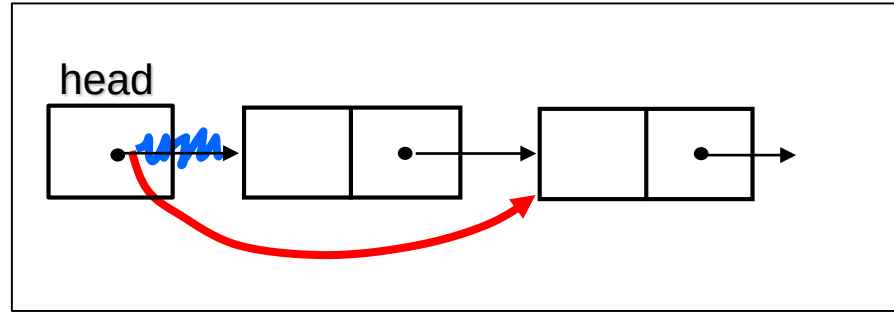


맨 끝 노드 삭제: Okay



pop()

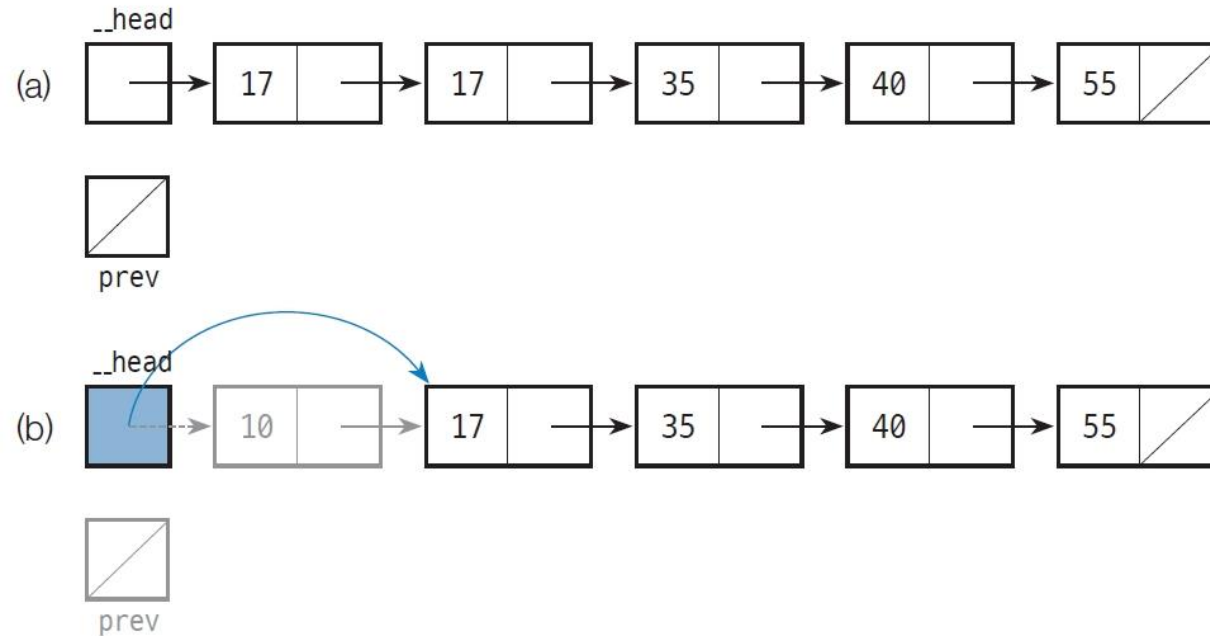
Animation



$\text{head} \leftarrow \text{head.next}$
 $\text{_numItems} -= 1$

맨 앞 노드를 삭제할 때

prev 가 존재하지 않아 prev.next 와 같은 표현 사용불가



알고리즘 5-3 연결 리스트의 원소 삭제하기

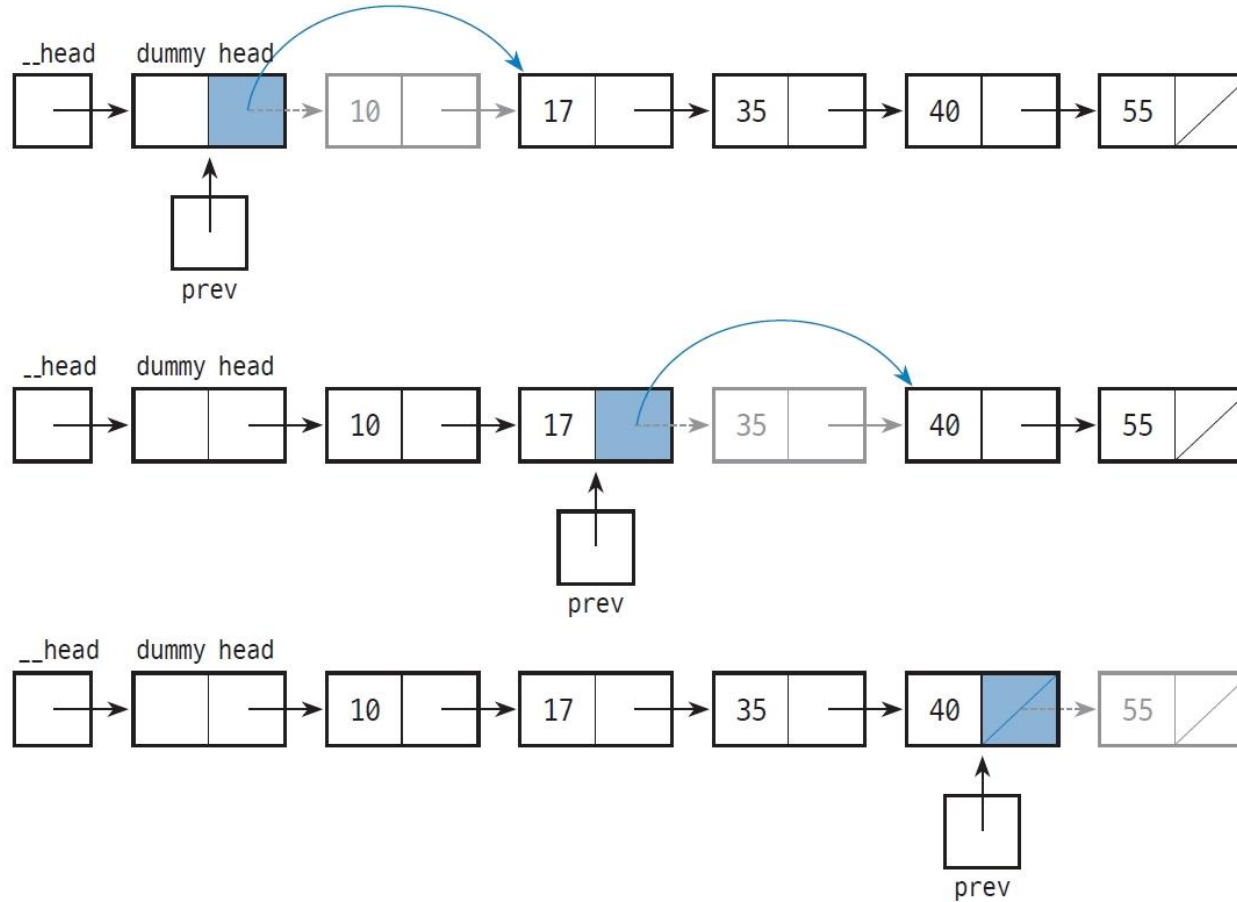
```
if i == 0:
    __head.next = __head.next.next
    __numItems -= 1
else:
    prev.next = prev.next.next
    __numItems -= 1
```

← 앞의 두 가지 경우를 다 고려한

자주 나오지 않는 “맨 앞 삭제” 때문에
이렇게 항상 두 가지 경우로 나누어 처리해야 하는가?

더미 헤드를 두면 역시 하나로 처리된다

pop()



알고리즘 5-4 연결 리스트의 원소 x 삭제하기(더미 헤드를 두는 대표 버전)

```
prev.next = prev.next.next
```

```
__numItems -= 1
```

← Dummy head를 두면 이것으로 충분
항상 `prev`가 존재하기 때문

기타 작업

알고리즘 5-5 연결 리스트의 i번 원소 알려주기

```
get(i):
    if i >= 0 and i <= __numItems - 1:
        return __getNode(i).item
    else:
        print("error in get(", i, ")")
```

알고리즘 5-6 연결 리스트의 i번 노드 알려주기

```
__getNode(i):
    curr = __head
    for index in range(i+1):
        curr = curr.next
    return curr
```

알고리즘 5-7 x가 연결 리스트의 몇 번 원소인지 알려주기

```
index(x):
    curr = __head # 더미 헤드
    for index in range(__numItems):
        curr = curr.next
        if curr.item == x:
            return index
    return -12345 # x가 없을 때
```

알고리즘 5-8 기타 작업들

```
isEmpty():          ◀ 리스트가 비었는지 알려주기
    return __numItems == 0

size():             ◀ 리스트의 총 원소 수 알려주기
    return __numItems

clear():            ◀ 리스트 깨끗이 청소하기
    newNode.next = None ◀ 더미 헤드 노드
    __head = newNode
    __numItems = 0
```

연결 리스트 클래스 구조

```
from DS.list.listNode import ListNode # 클래스 ListNode가 다른 파일에 있을 경우
```

```
class LinkedListBasic:
```

```
    def __init__(self):
```

```
        self.__head = ListNode('dummy', None)
```

```
        self.__numItems = 0
```

```
    ❶ def insert(self, i, newItem):
```

```
        ...
```

```
    ❷ def append(self, newItem):
```

```
        ...
```

```
    ❸ def pop(self, i):
```

```
        ...
```

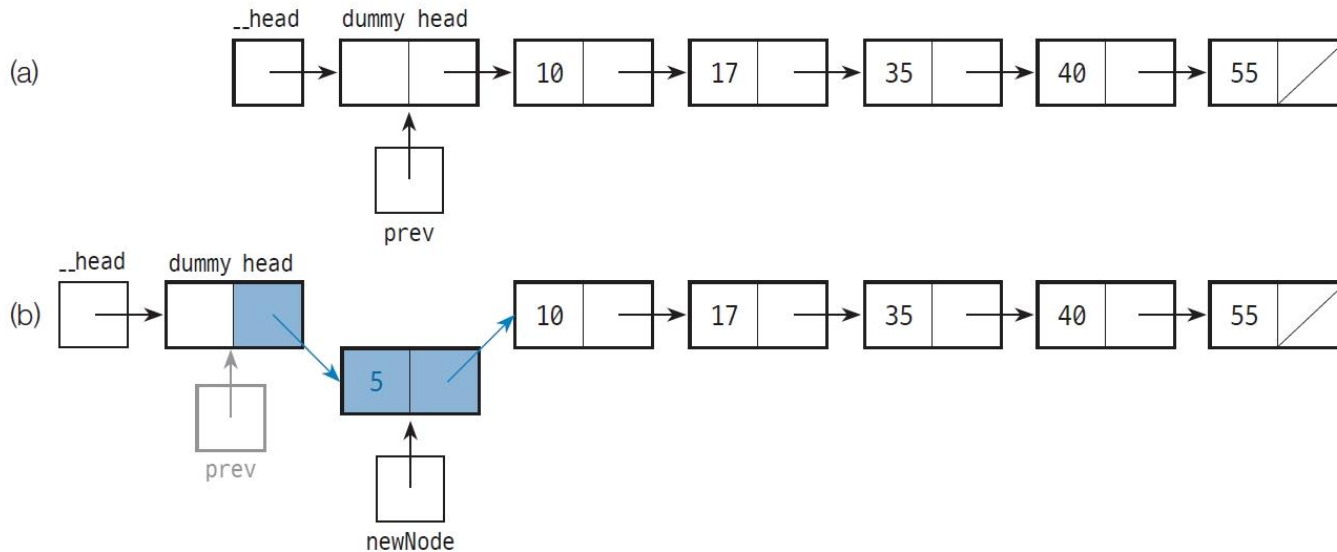
```
    ...
```

```

def insert(self, i:int, x):
    if i >= 0 and i <= self.__numItems:
        prev = self.__getNode(i - 1)
        newNode = ListNode(x, prev.next)
        prev.next = newNode
        self.__numItems += 1
    else:
        print("index", i, ": out of bound in insert()") # 필요 시 에러 처리
    
```

```

def __getNode(self, i:int) -> ListNode:
    curr = self.__head # dummy head, index: -1
    for index in range(i+1):
        curr = curr.next
    return curr
    
```



알고리즘 5-2 연결 리스트에 원소 삽입하기(더미 헤드를 두는 대표 버전)

```

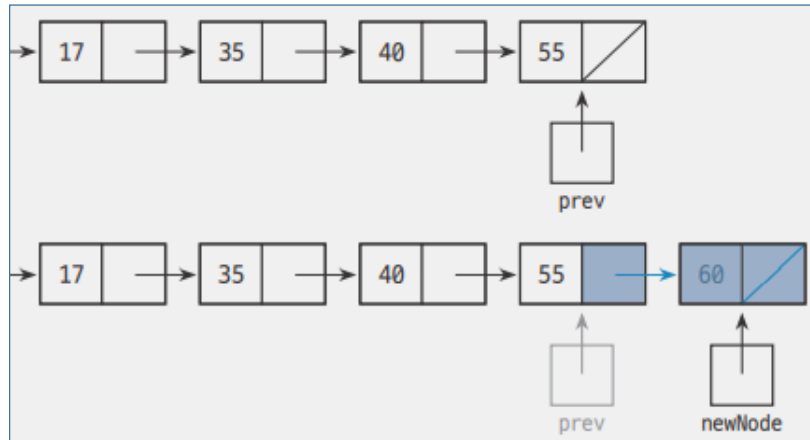
newNode.item = x
newNode.next = prev.next
prev.next = newNode
__numItems += 1
    
```

핵심부 (유사 코드)

함수 정의시 화살표(->)와 콜론(:) 의미

맨 끝 추가

```
def append(self, newItem):
    prev = self.__getNode(self.__numItems - 1)
    newNode = ListNode(newItem, prev.next)
    prev.next = newNode
    self.__numItems += 1
```



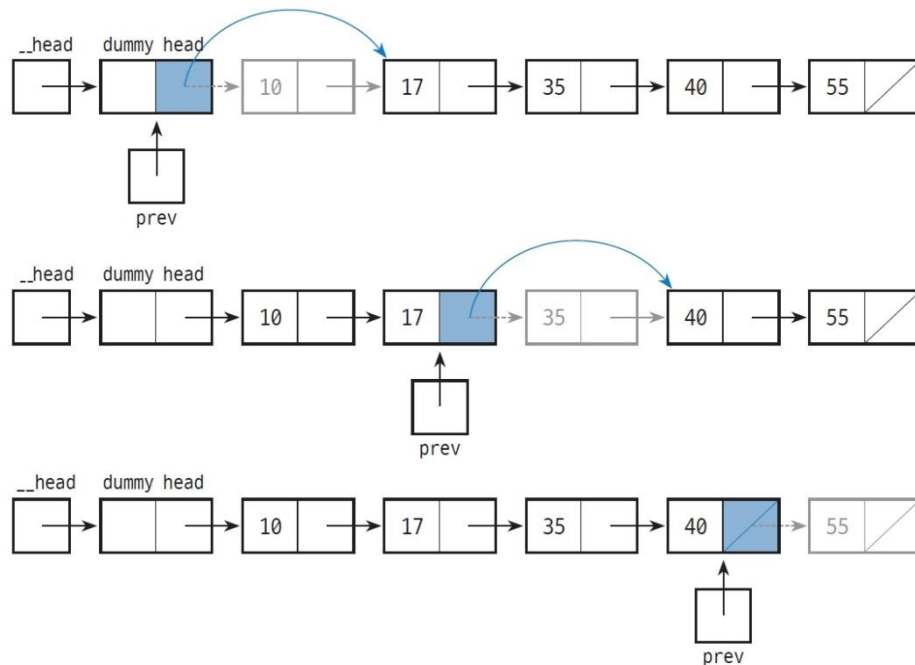
알고리즘 5-2 연결 리스트에 원소 삽입하기(더미 헤드를 두는 대표 버전)

```
newNode.item = x
newNode.next = prev.next
prev.next = newNode
__numItems += 1
```

핵심부 (유사 코드)

```

def pop(self, i:int): # i번 노드 삭제
    if (i >= 0 and i <= self.__numItems-1):
        prev = self.__getNode(i - 1)
        curr = prev.next
        prev.next = curr.next
        retItem = curr.item
        self.__numItems -= 1
        return retItem
    else:
        return None
    
```



알고리즘 5-4 연결 리스트의 원소 x 삭제하기(더미 헤드를 두는 대표 버전)

```

prev.next = prev.next.next
__numItems -= 1
    
```

핵심부 (유사 코드)

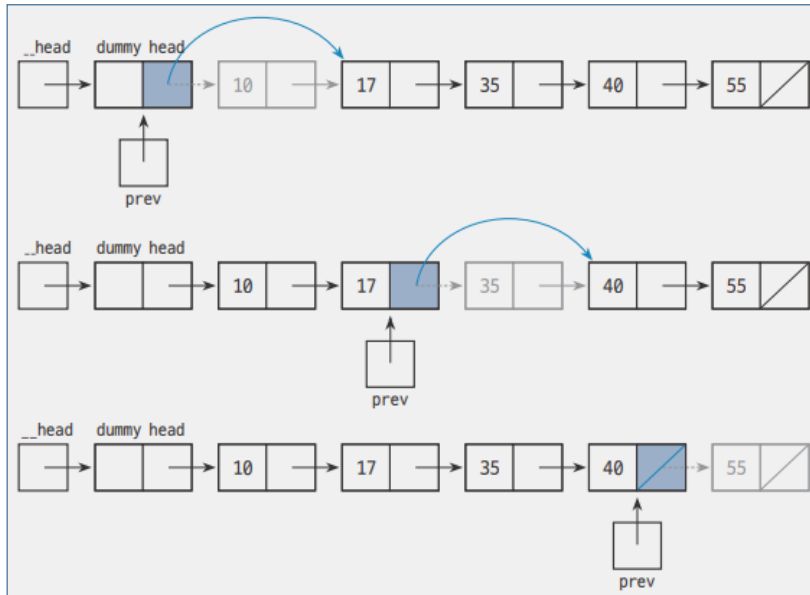
파이썬 코드

```

def remove(self, x):
    (prev, curr) = self.__findNode(x)
    if curr != None:
        prev.next = curr.next
        self.__numItems -= 1
    else:
        return None
    
```

```

def __findNode(self, i:int) -> (ListNode, ListNode):
    prev = self.__head # dummy head
    curr = prev.next # 0번 노드
    while curr != None:
        if curr.item == x:
            return (prev, curr)
        else:
            prev = curr; curr = curr.next
    return (None, None)
    
```



알고리즘 5-4 연결 리스트의 원소 x 삭제하기(더미 헤드를 두는 대표 버전)

```

prev.next = prev.next.next
__numItems -= 1
    
```

핵심부 (유사 코드)

파이썬 코드

```
def get(self, i:int):  
    if self.isEmpty():  
        return None  
    if (i >= 0 and i <= self.__numItems - 1):  
        return self.__getNode(i).item  
    else:  
        return None
```

```
def index(self, x) -> int:  
    curr = self.__head.next # 0번 노드: 더미 헤드 다음 노드  
    for index in range(self.__numItems):  
        if curr.item == x:  
            return index  
        else:  
            curr = curr.next  
    return -12345 # 안쓰는 인덱스
```

```
def isEmpty(self) -> bool:  
    return self.__numItems == 0
```

```
def size(self) -> int:  
    return self.__numItems
```

```
def clear(self):  
    self.__head = ListNode("dummy", None)  
    self.__numItems = 0
```

```
def count(self, x) -> int:  
    cnt = 0  
    curr = self.__head.next # 0번 노드  
    while curr != None:  
        if curr.item == x:  
            cnt += 1  
        curr = curr.next  
    return cnt
```

파이썬 코드



```
def extend(self, a): # 여기서 a는 self와 같은 타입의 리스트
    for index in range(a.size()):
        self.append(a.get(index))
```

```
def copy(self):
    a = LinkedListBasic()
    for index in range(self.__numItems):
        a.append(self.get(index))
    return a
```

```
def reverse(self):
    a = LinkedListBasic()
    for index in range(self.__numItems):
        a.insert(0, self.get(index))
    self.clear()
    for index in range(a.size()):
        self.append(a.get(index))
```

```
def sort(self) -> None:
    a = []
    for index in range(self.__numItems):
        a.append(self.get(index))
    a.sort()
    for index in range(len(a)):
        self.append(a[index])
```

```
def printList(self):
    curr = self.__head.next # 0번 노드: 더미 헤드 다음 노드
    while curr != None:
        print(curr.item, end=" ")
        curr = curr.next
    print()
```


listNode.py

```
class ListNode:
    def __init__(self, newItem, nextNode:'ListNode'):
        self.item = newItem
        self.next = nextNode
```

linkedListBasic.py

```
from listNode import ListNode

class LinkedListBasic:
    def __init__(self):
        self.__head = ListNode('dummy', None)
        self.__numItems = 0

    # [알고리즘 5 - 2] 구현: 연결 리스트에 원소 삽입하기(더미 헤드를 두는 대표 버전)
    def insert(self, i:int, newItem):
        if i >= 0 and i <= self.__numItems:
            prev = self.__getNode(i - 1)
            newNode = ListNode(newItem, prev.next)
            prev.next = newNode
            self.__numItems += 1
        else:
            print("index", i, ": out of bound in insert()") # 필요 시 에러 처리
```

```
def append(self, newItem):
    prev = self.__getNode(self.__numItems - 1)
    newNode = ListNode(newItem, prev.next)
    prev.next = newNode
    self.__numItems += 1

# [알고리즘 5-3] 구현: 연결 리스트의 원소 삭제하기
def pop(self, i:int): # i번 노드 삭제. 고정 파라미터
    if (i >= 0 and i <= self.__numItems-1):
        prev = self.__getNode(i - 1)
        curr = prev.next
        prev.next = curr.next
        retItem = curr.item
        self.__numItems -= 1
        return retItem
    else:
        return None
```

```
def remove(self, x):
    (prev, curr) = self.__findNode(x)
    if curr != None:
        prev.next = curr.next
        self.__numItems -= 1
        return x
    else:
        return None
```

[알고리즘 5 - 5] 구현: 연결 리스트의 i번 원소 알려주기

```
def get(self, i:int):
```

```
    if self.isEmpty():
        return None
    if (i >= 0 and i <= self.__numItems - 1):
        return self.__getNode(i).item
    else:
        return None
```

[알고리즘 5 -7] 구현: x가 연결 리스트의 몇 번 원소인지 알려주기

```
def index(self, x) -> int:
```

```
    curr = self.__head.next # 0번 노드: 더미 헤드 다음 노드
    for index in range(self.__numItems):
        if curr.item == x:
            return index
        else:
            curr = curr.next
    return -2 # 안 쓰는 인덱스
```

```
def isEmpty(self) -> bool:
    return self.__numItems == 0

def size(self) -> int:
    return self.__numItems

def clear(self):
    self.__head = ListNode("dummy", None)
    self.__numItems = 0

def count(self, x) -> int:
    cnt = 0
    curr = self.__head.next # 0번 노드
    while curr != None:
        if curr.item == x:
            cnt += 1
        curr = curr.next
    return cnt

def extend(self, a): # 여기서 a는 self와 같은 타입의 리스트
    for index in range(a.size()):
        self.append(a.get(index))
```

```
def copy(self):
    a = LinkedListBasic()
    for index in range(self.__numItems):
        a.append(self.get(index))
    return a

def reverse(self):
    a = LinkedListBasic()
    for index in range(self.__numItems):
        a.insert(0, self.get(index))
    self.clear()
    for index in range(a.size()):
        self.append(a.get(index))

def sort(self) -> None:
    a = []
    for index in range(self.__numItems):
        a.append(self.get(index))
    a.sort()
    self.clear()
    for index in range(len(a)):
        self.append(a[index])
```

Source

```
def __findNode(self, x) -> (ListNode, ListNode):
    prev = self.__head # 더미 헤드
    curr = prev.next    # 0번 노드
    while curr != None:
        if curr.item == x:
            return (prev, curr)
        else:
            prev = curr; curr = curr.next
    return (None, None)

# [알고리즘 5-6] 구현: 연결 리스트의 i번 노드 알려주기
def __getNode(self, i:int) -> ListNode:
    curr = self.__head # 더미 헤드, index: -1
    for index in range(i+1):
        curr = curr.next
    return curr

def printList(self):
    curr = self.__head.next # 0번 노드: 더미 헤드 다음 노드
    while curr != None:
        print(curr.item, end = ' ')
        curr = curr.next
    print()
```

Source

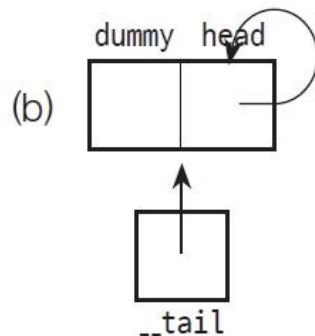
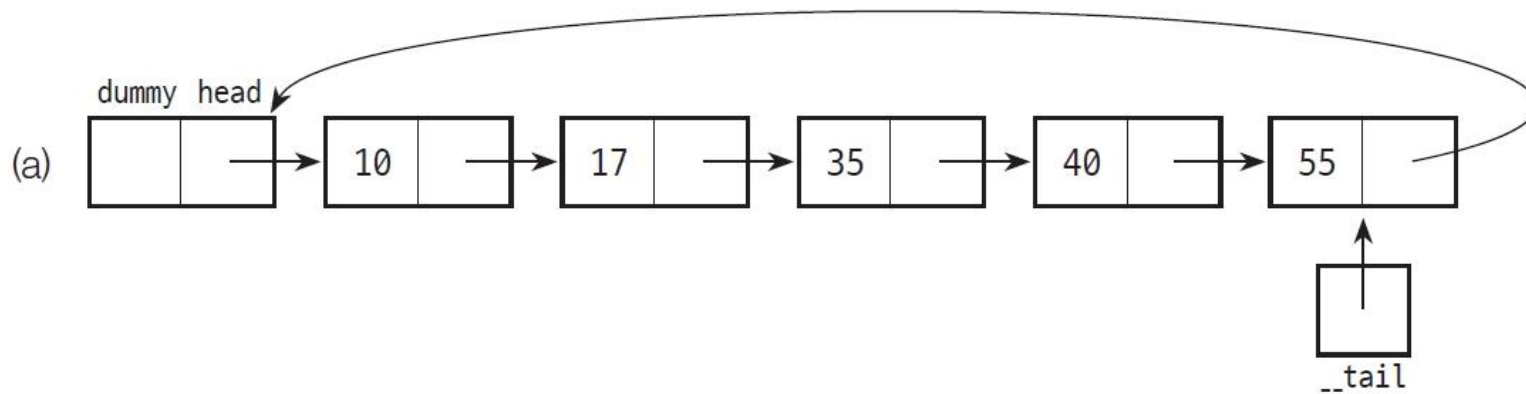
linkedListBasicDemo.py

```
from linkedListBasic import LinkedListBasic

list = LinkedListBasic()
list.append(30); list.insert(0, 20)
a = LinkedListBasic()
a.append(4); a.append(3); a.append(3); a.append(2); a.append(1)
list.extend(a)
list.reverse()
list.pop(0)
print("count(3):", list.count(3))
print("get(2):", list.get(2))
list.printList()
```

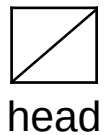
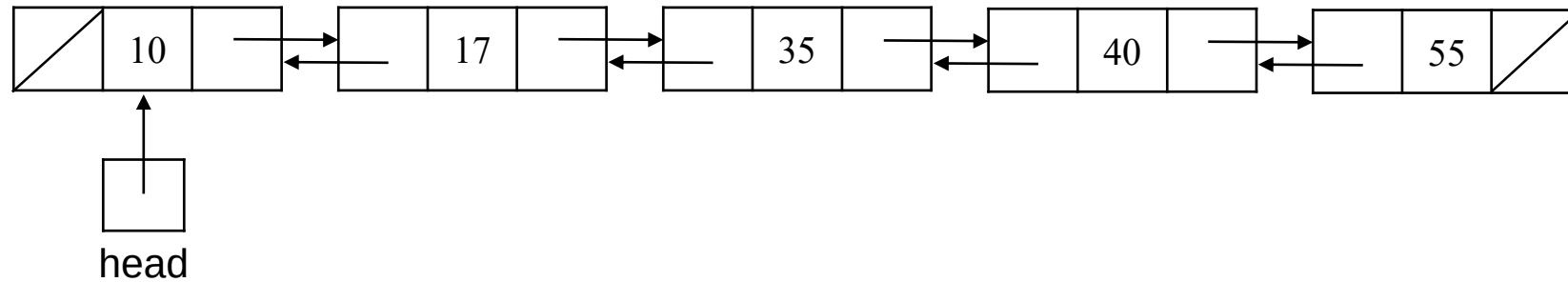
원형 연결 리스트 Circular Linked List

끝 노드(tail)의 next가 null 값을 갖는 대신 첫 노드를 링크한다
 맨 앞과 맨 뒤의 접근성 차이가 없어짐



양방향 연결 리스트 Doubly Linked List

양방향 연결 리스트의 예



초기 상태: Empty list

노드 구조



```
class BidirectNode:
    def __init__(self, newItem, prevNode:'BidirectNode', nextNode:'BidirectNode'):
        self.item = newItem
        self.prev = prevNode
        self.next = nextNode
```

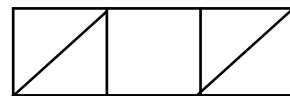
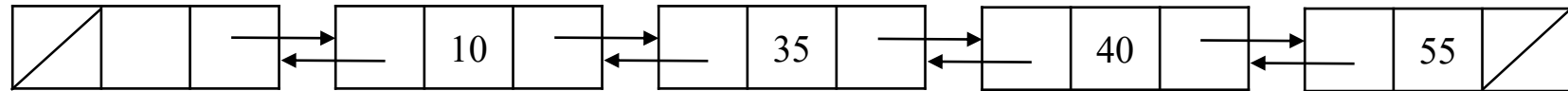

더미 헤드를 가진 양방향 연결 리스트

삽입

```
newNode ← BidirectNode(newItem, prev, prev.next)
newNode.next.prev ← newNode
prev.next ← newNode
numItems++
```

삭제

```
curr.prev.next ← curr.next
curr.next.prev ← curr.prev
numItems--
```

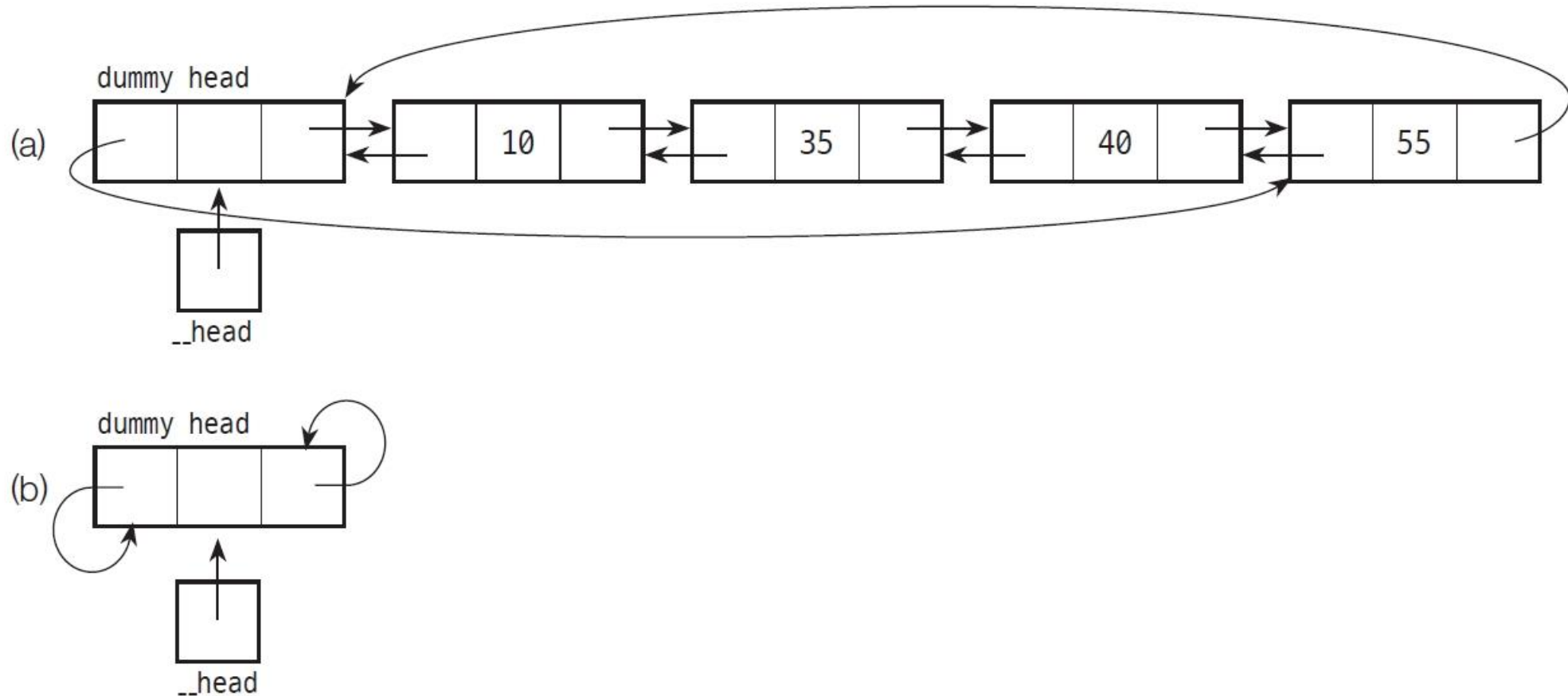


초기 상태: Empty list

- ✓ 중간, 맨 앞을 구분할 필요 없다
- ✓ 맨 앞 작업도 중간 작업과 동일

더미 헤드를 가진 원형 양방향 연결 리스트

연결 리스트의 모든 연결 방식이 포함된 버전



Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- <https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>
- https://www.w3schools.com/python/python_ref_list.asp
- <https://www.geeksforgeeks.org/list-methods-python/>
- <https://wikidocs.net/16036>
- <https://drive.google.com/drive/folders/1XXNIpQFYNRJ3xRIMYg7kz6zNIOATMqZm>